Services and APIs migration to new AKS cluster with ArgoCD setup

1. Eliminate Secrets from CICD Pipeline Ensure that all secrets are removed from your CICD pipeline and securely transferred to their respective project vaults.
2. **Create Managed Identity Resource with Federated Credential** To create a managed identity resource and assign permissions to the vaults using Terraform, follow the example below:

   (Below changes are already done on Dev and Intg by running keyVault pipeline)

WrittentNotification.KeyVault.Infra (P2P-Own Subscription)(YML) : https://dev.azure.com/eviCoreDev/eviCore Platform/_build/results?buildId=799853&view=results

```
resource "azurerm_user_assigned_identity" "wns_user_assigned_identity" {
  depends_on          = [module.golden_key_vault_module]
  name                = "${var.environment}-wns-user-assigned-identity"
  tags                = var.tags
  resource_group_name = data.azurerm_resource_group.resource_group.name
  location            = var.location
}

resource "azurerm_key_vault_access_policy" "key_vault_access_policy" {
  depends_on   = [azurerm_user_assigned_identity.wns_user_assigned_identity]
  key_vault_id = module.golden_key_vault_module.key_vault_id
  tenant_id    = data.azurerm_client_config.current.tenant_id
  object_id    = azurerm_user_assigned_identity.wns_user_assigned_identity.principal_id
  secret_permissions = [
    "Get",
    "List",
  ]
}

resource "azurerm_federated_identity_credential" "wns-federated-credential" {
  depends_on          = [azurerm_user_assigned_identity.wns_user_assigned_identity]
  name                = "${var.environment}-wns-federated-credential"
  resource_group_name = data.azurerm_resource_group.resource_group.name
  parent_id           = azurerm_user_assigned_identity.wns_user_assigned_identity.id
  audience            = ["api://AzureADTokenExchange"]
  issuer              = "${var.aks_cluster_issuer}"
  subject             = "system:serviceaccount:${var.aks_namespace}:${var.sa_name}"
}
```

3. **Adding a Secret Provider YAML**

To add a Secret Provider YAML, follow these steps:

Include the Secret Name: In the object array provided in the code snippet, include the secret name that you created in Vault as mentioned in step #1.

Case Sensitivity: Please note that `objectName` is case-sensitive.

Matching Object Names: Ensure that the `objectName` in the “objects” section matches exactly with the `objectName` in the “secretObjects” section.

Key Vault Name: The `keyvaultName` should correspond to the name of your vault.

By adhering to these guidelines, you can successfully configure your Secret Provider YAML.

secretprovider.yml

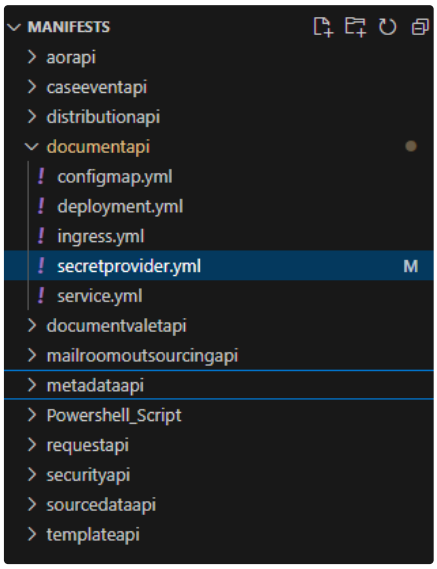
```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: wns-workload-identity-sa
  namespace: __AksNamespace__
  annotations:
    azure.workload.identity/client-id: __UserAssignedIdentityClientId__

---

apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: wns-azure-kv-secret-provider
  namespace: __AksNamespace__
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    clientID: __UserAssignedIdentityClientId__
    keyvaultName: __WNSAkv__
    objects: |
      array:
        - |
          objectName: APIMSubscriptionKey
          objectType: secret
        - |
          objectName: MailOutSourcingServiceUsername
          objectType: secret
        - |
          objectName: MailOutSourcingServicePassword
          objectType: secret
        - |
          objectName: ServiceBusConnectionString
          objectType: secret
    tenantId: __TenantId__
  secretObjects:
  - secretName: apimsubscriptionkey
    type: Opaque
    data:
    - key: apimsubscriptionkey
      objectName: APIMSubscriptionKey
  - secretName: saslusername
    type: Opaque
    data:
    - key: saslusername
      objectName: MailOutSourcingServiceUsername
  - secretName: saslpassword
    type: Opaque
    data:
    - key: saslpassword
      objectName: MailOutSourcingServicePassword
```

```
  - secretName: servicebusconnectionstring
    type: Opaque
    data:
    - key: servicebusconnectionstring
      objectName: ServiceBusConnectionString
```

Add secretprovider.yml file for every Service and API in respective manifest folder as per below



4. **Addition in Deployment.yml** deployment.yml

Add the yellow-marked labels and spec in your deployment. yaml for accessing the vault via MI

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mailroomoutsourcingservice
  labels:
    type: service
spec:
  replicas: __Replicas__
  selector:
    matchLabels:
      app: mailroomoutsourcingservice
  template:
    metadata:
      labels:
        azure.workload.identity/use: "true"
        app: mailroomoutsourcingservice
    spec:
      serviceAccountName: wns-workload-identity-sa
      containers:
        - image: __WNMailroomOutsourcingServiceContainerImage__
          imagePullPolicy: Always
```

Add your secret as an env variable in deployment.yaml for accessing secrets in c# in below manner

```yaml
        env:
          - name: Apim__SubscriptionKey
            valueFrom:
              secretKeyRef:
                name: apimsubscriptionkey
                key: apimsubscriptionkey
          - name: BusV2__Kafka__SaslUsername
            valueFrom:
              secretKeyRef:
                name: saslusername
                key: saslusername
          - name: BusV2__Kafka__SaslPassword
            valueFrom:
              secretKeyRef:
                name: saslpassword
                key: saslpassword
          - name: AzureServiceBus__ConnectionString
            valueFrom:
              secretKeyRef:
                name: servicebusconnectionstring
                key: servicebusconnectionstring
        volumeMounts:
          - name: config-volume
            mountPath: /app/config
          - name: secret-volume
            mountPath: /app/secret
      volumes:
        - name: secret-volume
          csi:
            driver: secrets-store.csi.k8s.io
            readOnly: true
            volumeAttributes:
              secretProviderClass: "wns-azure-kv-secret-provider"
```

5. **Changes in Ingress.yaml if your app is an API.** ingress.yml

Add the yellow-marked values in your ingress.yaml for accurate attribute routing accuracy

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: wn-documentapi-ingress
  namespace: __AksNamespace__
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.org/mergeable-ingress-type: "minion"
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/x-forwarded-prefix: /documentapi
    nginx.org/client-max-body-size: "100m"
spec:
  rules:
    - host: __ArgoCDAKSCluster__
      http:
        paths:
          - path: "/documentapi(/|$)(.*)"
            pathType: Prefix
            backend:
              service:
                name: documentapi
                port:
                  number: 80 # Requests coming to above path is redirecte
```

Pipeline Changes

6. Remove/Comment Azure Key Vault, Powershell tokenization and `kubectl` deployment tasks from generic-deployment.yml or Release pipeline.

7. Add ArgoCD Powershell script task into generic-deployment.yml or Release pipeline for new AKS cluster deployment.

(ArgoCD Powershell script task already added for Services and APIs project)

```yaml
parameters:
  AZURE_ENVIRONMENT: ""
  SERVICE_PRINCIPAL: ""
  ENVIRONMENT_NAME: ""
  DEPENDS_ON: ""
  ARTIFACT_FOLDER: ""
  PS_TOKENIZATION_PARAMETER: ""
  KEY_VALUT_NAME: ""
  PROJECT_ALICE: ""
  AKS_SERVICE_END_POINT: ""
  PS_ARGOCD_PARAMETER: ""
  ARGOCD_PAT: ""
  VARIABLE_GROUPS: {}

stages:
  - stage: ${{ parameters.ENVIRONMENT_NAME }}
    displayName: ${{ parameters.ENVIRONMENT_NAME }}
    dependsOn: ${{ parameters.DEPENDS_ON  }}
    pool:
      name: "eheu2pd1epvmss"
    variables:
      - ${{ each groupname in parameters.VARIABLE_GROUPS  }}:
          - group: ${{groupname}}
    jobs:
      - deployment: "Deployment"
        environment: ${{ parameters.AZURE_ENVIRONMENT  }}
        strategy:
          runOnce:
            deploy:
              steps:
                - task: DownloadBuildArtifacts@0
                  inputs:
                    buildType: "current"
                    downloadType: "single"
                    artifactName: "Manifests"
                    downloadPath: "$(System.DefaultWorkingDirectory)/${{parameters.ARTIFACT_FOLDER}}"

                # - task: AzureKeyVault@2
                #   displayName: "Azure Key Vault: ${{ parameters.KEY_VALUT_NAME }}"
                #   inputs:
                #     azureSubscription: ${{ parameters.SERVICE_PRINCIPAL }}
                #     KeyVaultName: ${{ parameters.KEY_VALUT_NAME }}
                #     RunAsPreJob: true

                # - task: PowerShell@2
                #   displayName: "PowerShell Keyvault Tokenization Script"
                #   inputs:
                #     targetType: filePath
                #     filePath: "$(System.DefaultWorkingDirectory)/${{parameters.ARTIFACT_FOLDER}}/Manifests/Powershell_Script/TokenizationScript.ps1"
                #     arguments: ${{parameters.PS_TOKENIZATION_PARAMETER}}

                - task: TotalALM.totalalm-tokenization.tokenization-build-task.Tokenization@2
                  displayName: "Tokenization: Transform file *.yml"
                  inputs:
                    SourcePath: "$(System.DefaultWorkingDirectory)"
                    TargetFileNames: "*.yml"
```

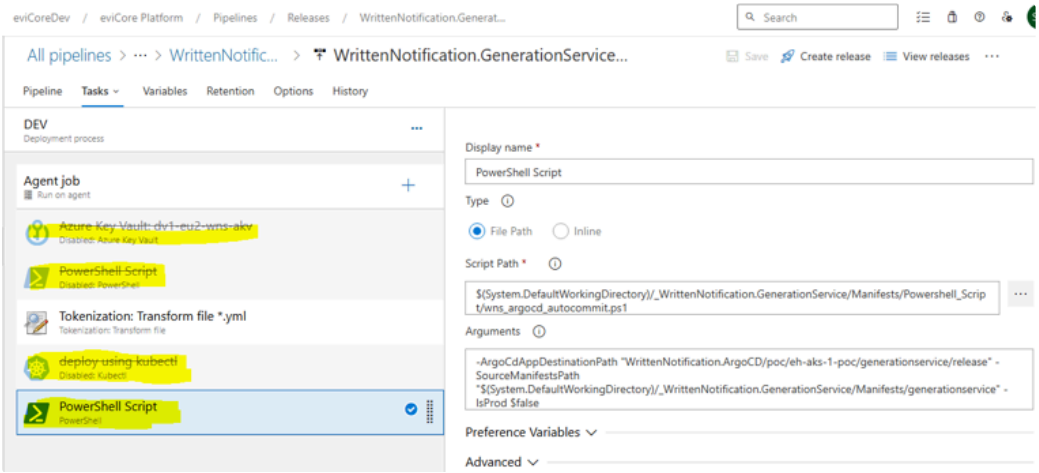
```

                # - task: Kubernetes@1
                #   displayName: "deploy using kubectl"
                #   inputs:
                #     kubernetesServiceEndpoint: ${{ parameters.AKS_SERVICE_END_POINT }}
                #     namespace: writtennotification
                #     command: apply
                #     useConfigurationFile: true
                #     configuration: "$(System.DefaultWorkingDirectory)/${{parameters.ARTIFACT_FOLDER}}/Manifests/${{parameters.PROJECT_ALICE}}"
                #     azureSubscriptionEndpointForSecrets: "sp_eh-writtennotifications-dev"
                #     azureContainerRegistry: dv1eu2wnsacr.azurecr.io
                #     secretName: wnsecret
                #     configMapName: "wn-${{parameters.PROJECT_ALICE}}-config"
                #     forceUpdateConfigMap: true

                - task: PowerShell@2
                  displayName: "ArgoCD PowerShell Script"
                  env:
                    PAT: '60JnwF2D9MtiZyVY5Ql3x8SoJVPNNPkTL89SmRCHMWnG8nEYmTOnJQQJ99BCACAAAAAkkYojAAASAZDODVke'
                  inputs:
                    targetType: filePath
                    filePath: "$(System.DefaultWorkingDirectory)/${{parameters.ARTIFACT_FOLDER}}/Manifests/Powershell_Script/wns_argocd_autocommit.ps1"
                    arguments: ${{parameters.PS_ARGOCD_PARAMETER}}

```

For task based deployment



8 Add below WNS ArgoCD autocommit powershell script to Manifests/Powershell_Script folder.

(ArgoCD Powershell script already added for Services and APIs project)

wns_argocd_au...mit.ps1
11 Apr 2025, 11:07 AM

9 Pass argument to ArgoCD Powershell script as follow

`PS_ARGOCD_PARAMETER: '-ArgoCdAppDestinationPath "WrittenNotification.ArgoCD/dev/eh-aks-1-dev/services/mailroomoutsourcingservice/release" -SourceManifestsPath "$(System.DefaultWorkingDirectory)/$(ARTIFACT_FOLDER)/Manifests/mailroomoutsourcingservice" -IsProd $false'`

`ArgoCdAppDestinationPath` : Service or API path of `WrittenNotification.ArgoCD` repo as per environment

`SourceManifestsPath` : Manifest files folder path of Service or API

10 Add EnvironmentVariables to configuration builder as per below.

```
public static IServiceCollection RegisterConfiguration(this IServiceCollection services)
{
    var builder = new ConfigurationBuilder()
        .AddJsonFile($"appsettings.Development.json", true, true)
        .AddJsonFile($"config/appsettings.json", true, true)
        .AddEnvironmentVariables();
    var configuration = builder.Build();
    services.AddSingleton(typeof(IConfiguration), configuration);
    return services;
}
```

Key Vault Secrets would be added to environment variables of Pods

11 Add below code for Swagger configuration

```
services.Configure<ForwardedHeadersOptions>(options =>
{
    options.ForwardedHeaders = ForwardedHeaders.XForwardedFor | ForwardedHeaders.XForwardedProto;
});
```

```
app.UsePathBase("/documentapi");
app.UseForwardedHeaders();

if (!env.EnvironmentName.Equals("Production"))
{
    app.UseDeveloperExceptionPage();
    app.UseSwagger(c =>
    {
        c.PreSerializeFilters.Add((swaggerDoc, request) =>
        {
            if (request.Headers.TryGetValue("X-Forwarded-Prefix", out var serverPath))
            {
                swaggerDoc.Servers = new List<OpenApiServer>() { new OpenApiServer() { Url = serverPath } };
            }
        });
    });
    app.UseSwaggerUI(options =>
    {
        options.SwaggerEndpoint("./v1/swagger.json", "DocumentAPI");
```

```
    });
}
```